

CSE 482 Spring 2026 Final Project Report

Movie Recommendation System using Collaborative Filtering

Team Members and Roles

Team Member: Mehrshad Bagherebadian

Note: This project was completed individually.

Role Area	Work Completed
Data collection and preprocessing	Downloaded and organized the MovieLens latest-small dataset, inspected data quality, cleaned tables, converted timestamps, merged rating and movie metadata, and prepared genre features.
Database and container setup	Stored cleaned tables in a MySQL database running through Docker and verified table counts and SQL queries.
Data analysis and modeling	Implemented baseline, user-based collaborative filtering, and item-based collaborative filtering models with train/test evaluation.
Web application development	Built a React + TypeScript frontend, Flask backend API, MySQL database connection, and Docker Compose setup.
Reporting and interpretation	Prepared visualizations, compared model performance, summarized lessons learned, and documented the final system.

Project Deployment Link

Google Cloud Deployment Link: <https://cse482-frontend-816507479795.us-central1.run.app/>

This section is included for the final deployment link. The application was developed as a full-stack web application with React + TypeScript, Flask, MySQL, and Docker. The frontend is structured so it can be deployed separately after the backend/API deployment is finalized.

1. Problem Definition

The goal of this project was to develop a movie recommendation system using the MovieLens latest-small dataset. The main problem was to predict what movies a user may like based on previous ratings and then present those recommendations in a working web application. The project was designed around the recommender system option, which required user-based and item-based recommendation methods, train/test evaluation, and a web interface.

The specific questions addressed by the project were:

- Can user-movie rating data be used to recommend movies that a user has not rated yet?
- Which approach performs better on the test set: user-based collaborative filtering or item-based collaborative filtering?
- How do sparsity, user activity, movie popularity, and genre distribution affect recommendation quality?

- Can the model outputs be connected to a web interface that supports login/logout, rated movie display, recommendations, and similar-user display?

The final system is therefore both a data analysis project and a small software engineering project. It starts with raw rating data, stores the processed data in a database, trains recommendation models, and exposes the results through a browser-based dashboard.

2. Data Collection and Storage

The project used the MovieLens latest-small dataset from GroupLens Research. This dataset was selected because it is designed for recommender system development and is small enough to support rapid experimentation while still containing enough ratings to evaluate collaborative filtering methods.

Dataset Detail	Value
Ratings	100,836
Users	610
Movies	9,742 total movies; 9,724 rated movies
Rating scale	0.5 to 5.0 stars
Files used	ratings.csv, movies.csv, tags.csv, links.csv
Primary model files	ratings.csv and movies.csv
Storage	MySQL database running in Docker

The original data was provided as CSV files. The main modeling table was ratings.csv, which contains userId, movieId, rating, and timestamp. The movies.csv file provided movie titles and genre metadata. The tags and links files were inspected for completeness, but the main recommendation models were driven by ratings and movie metadata.

After preprocessing, the cleaned data was stored in a MySQL database named cse482_movielens. The database was run locally through Docker, which made the system easier to reproduce and connect to the backend application.

Database Table	Purpose
ratings	Cleaned user-movie rating interactions with converted datetime values.
movies	Movie metadata including movieId, title, and genres.
movies_genres	Processed movie table with one-hot encoded genre columns.
ratings_with_movies	Merged table used for display, analysis, and web application output.

3. Data Preprocessing

The MovieLens dataset was relatively clean, but preprocessing was still necessary before model training. The work focused on data quality checks, consistency checks, feature preparation, and transforming the data into a structure that could support collaborative filtering.

- Checked missing values in ratings, movies, tags, and links.

- Confirmed no missing values in ratings, movies, or tags.
- Found 8 missing tmdbId values in links.csv; this file was not used for modeling.
- Checked duplicate rows and confirmed none were present in the main files.
- Verified that all rating values were valid within the 0.5 to 5.0 rating scale.
- Confirmed that all rated movie IDs existed in movies.csv.
- Identified 18 movies in movies.csv that had no ratings.
- Converted Unix timestamps to readable datetime values.
- Merged ratings with movie metadata so model outputs could be displayed with titles and genres.
- Created 20 one-hot encoded genre columns for analysis and possible future hybrid modeling.

A major finding from preprocessing was that the user-item matrix is highly sparse. The project had 610 users and 9,724 rated movies, which gives 5,931,640 possible user-movie pairs. Only 100,836 ratings were observed. This means the matrix density was about 0.017 and the sparsity was about 0.983. This sparsity later became one of the main reasons item-based collaborative filtering performed better than user-based collaborative filtering.

4. Data Exploration and Visualization

Several visualizations were created to better understand the rating behavior, user activity, movie popularity, genre composition, and model performance. These figures were important because they explained why some recommendation methods performed better than others.

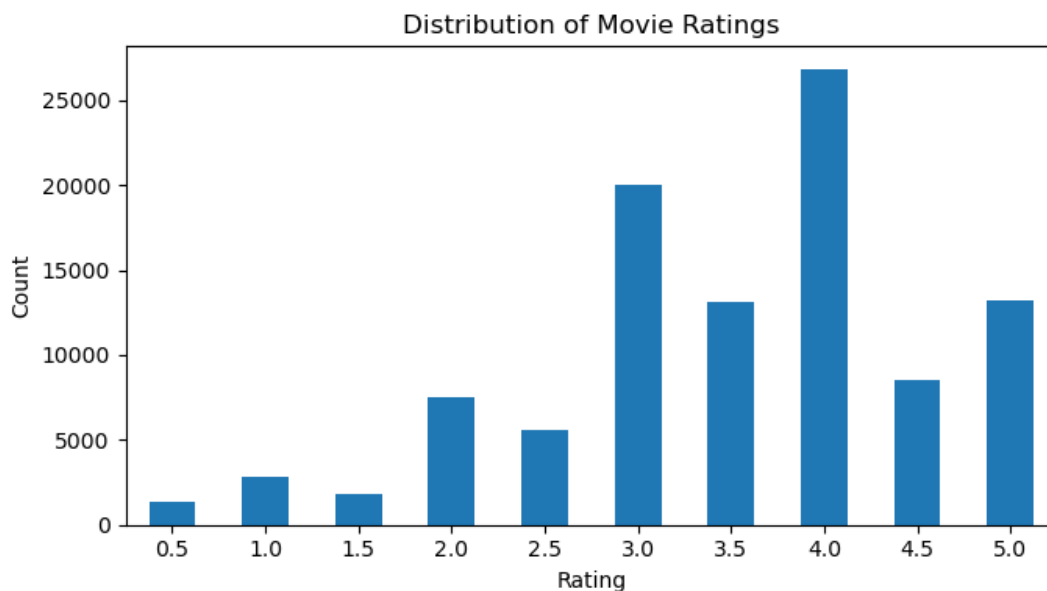


Figure 1. Distribution of movie ratings.

Most ratings are concentrated around 3.0 and 4.0, showing that the dataset is slightly positive overall. This matters because a model can appear reasonable even when predictions stay near the middle-to-high rating range.

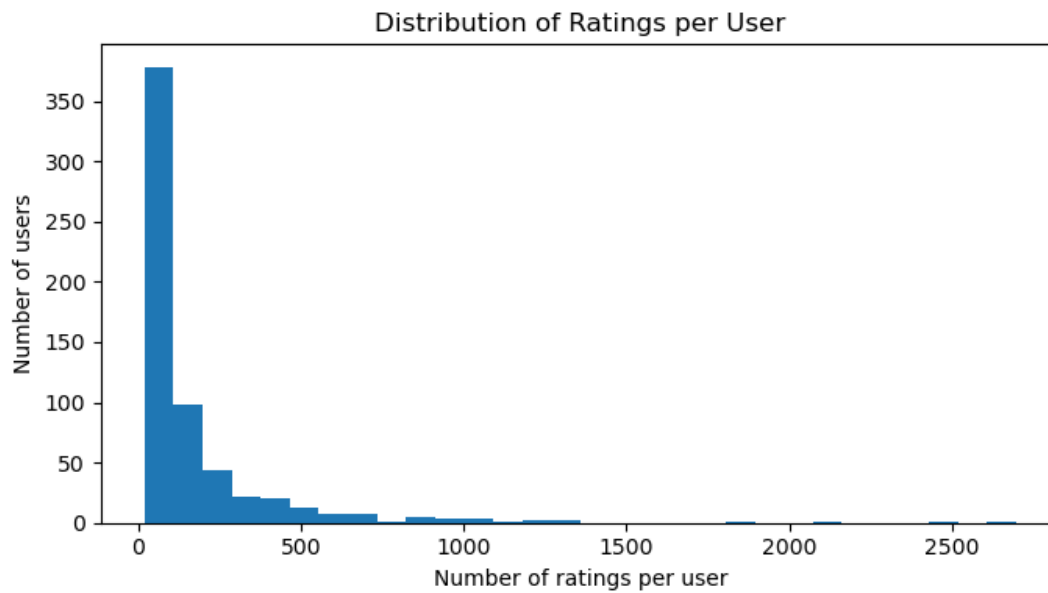


Figure 2. Distribution of ratings per user.

User activity is highly skewed. Most users rated a relatively small number of movies, while a few users rated hundreds or thousands. This makes user-user similarity harder because many users have limited overlap.

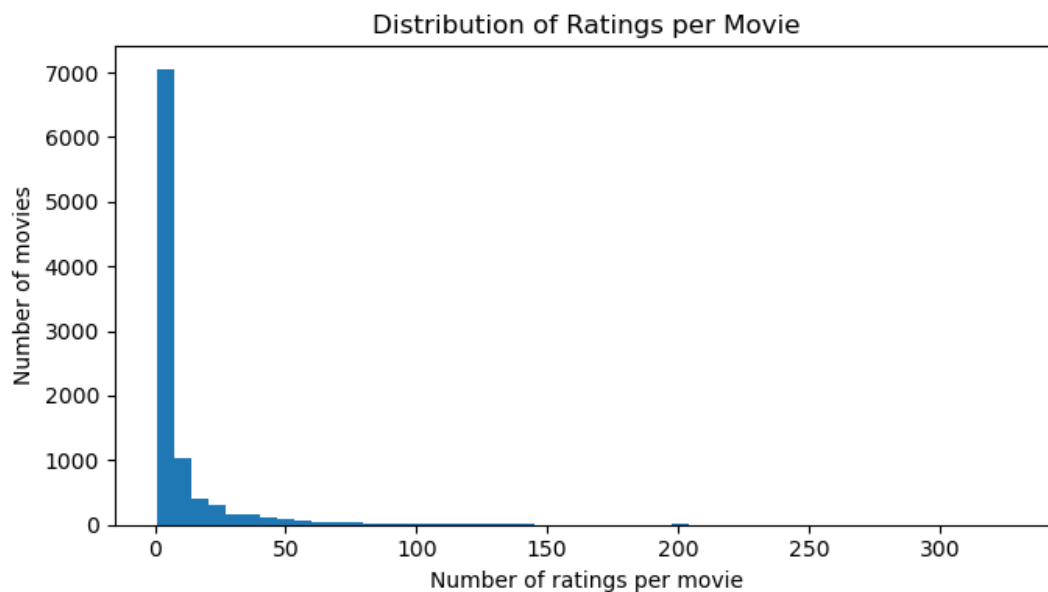


Figure 3. Distribution of ratings per movie.

Movie popularity is also highly skewed. Most movies have few ratings, while a small group of popular movies has many ratings. This affects both training efficiency and similarity reliability.

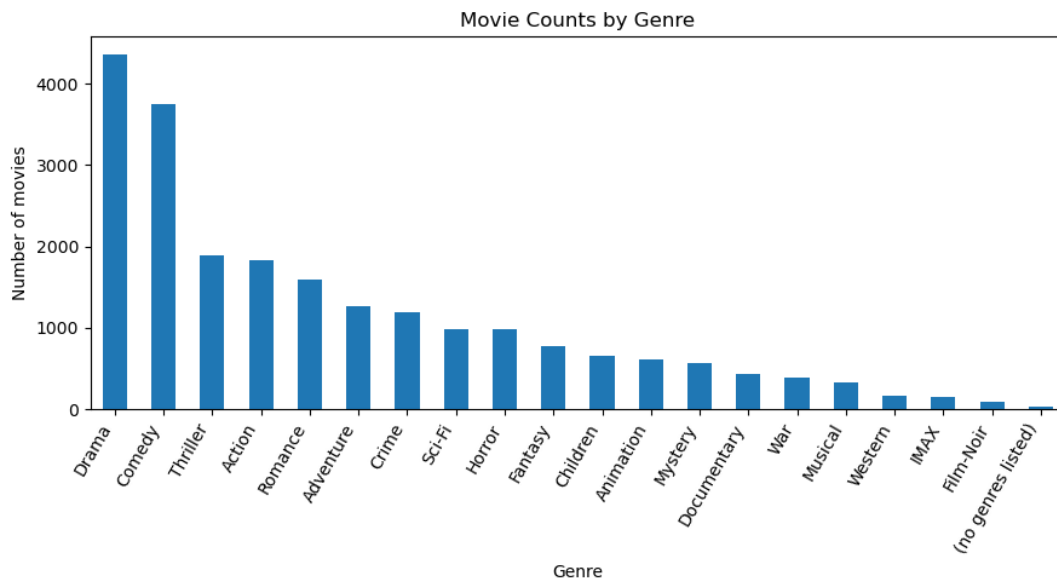


Figure 4. Movie counts by genre.

Drama, Comedy, Thriller, Action, and Romance are the most common genres. Genre information was used for interpretation and display, and it could support future hybrid recommendation models.

5. Data Analysis and Model Training

The cleaned ratings data was split into training and test sets using an 80/20 split. The training set contained 80,668 ratings and the test set contained 20,168 ratings. All 610 users were represented in both the training and test sets. Similarity calculations were based only on the training data to avoid leaking test information into the model.

Three models were compared:

- Baseline user-mean model: predicts a user average rating when available.
- User-based collaborative filtering: finds top-k similar users and predicts ratings using weighted averages from neighboring users.
- Item-based collaborative filtering: finds top-k similar movies and predicts ratings using weighted averages from similar movies the user has already rated.

The neighborhood size was tuned from $k = 50$ to $k = 100$. Increasing k improved both collaborative filtering models, especially the item-based model. The final system uses $k = 100$.

Model	RMSE	MAE
Baseline User Mean	0.9413	0.7347
User-Based CF	0.9676	0.7476
Item-Based CF	0.9016	0.6801

The item-based collaborative filtering model achieved the best performance with $RMSE = 0.9016$ and $MAE = 0.6801$. The baseline model had $RMSE = 0.9413$ and $MAE = 0.7347$. The user-based collaborative filtering model had $RMSE = 0.9676$ and $MAE = 0.7476$, which means it did not outperform the baseline.

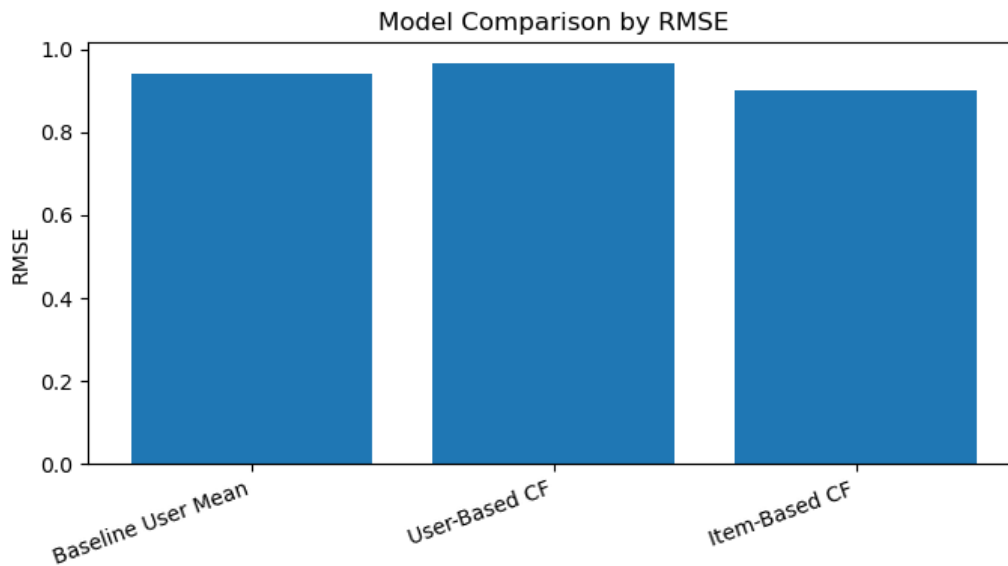


Figure 5. Model comparison by RMSE.

Lower RMSE indicates better predictive performance. Item-based collaborative filtering had the lowest RMSE.

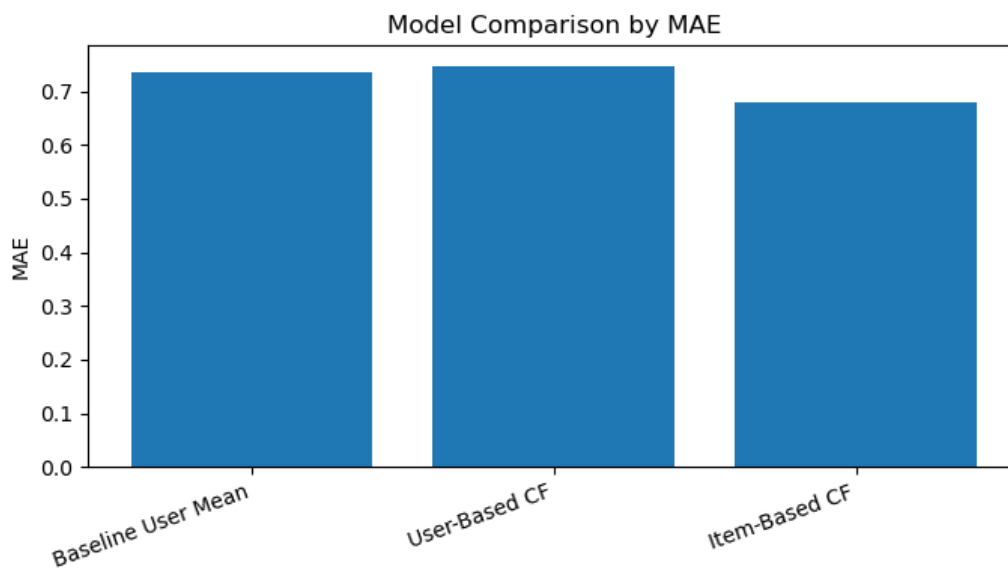


Figure 6. Model comparison by MAE.

MAE showed the same pattern as RMSE. Item-based collaborative filtering had the lowest average absolute error.

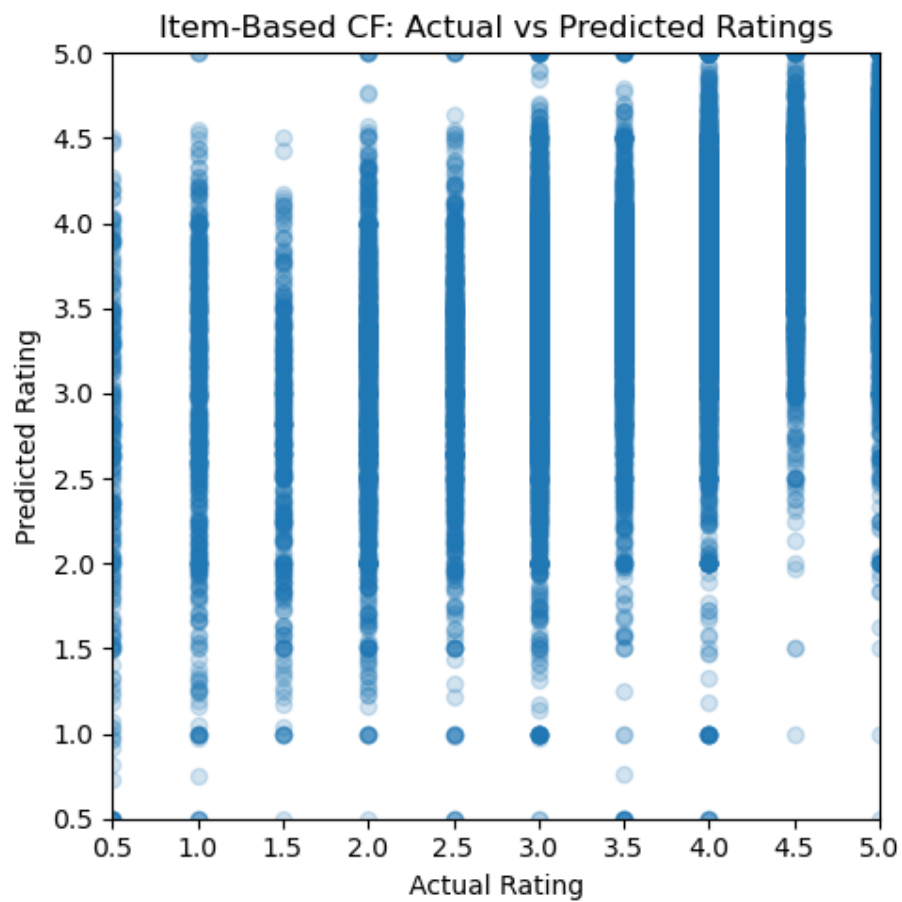


Figure 7. Item-based collaborative filtering: actual vs. predicted ratings.

The item-based predictions stay within the valid rating range and are more reliable than the user-based approach overall.

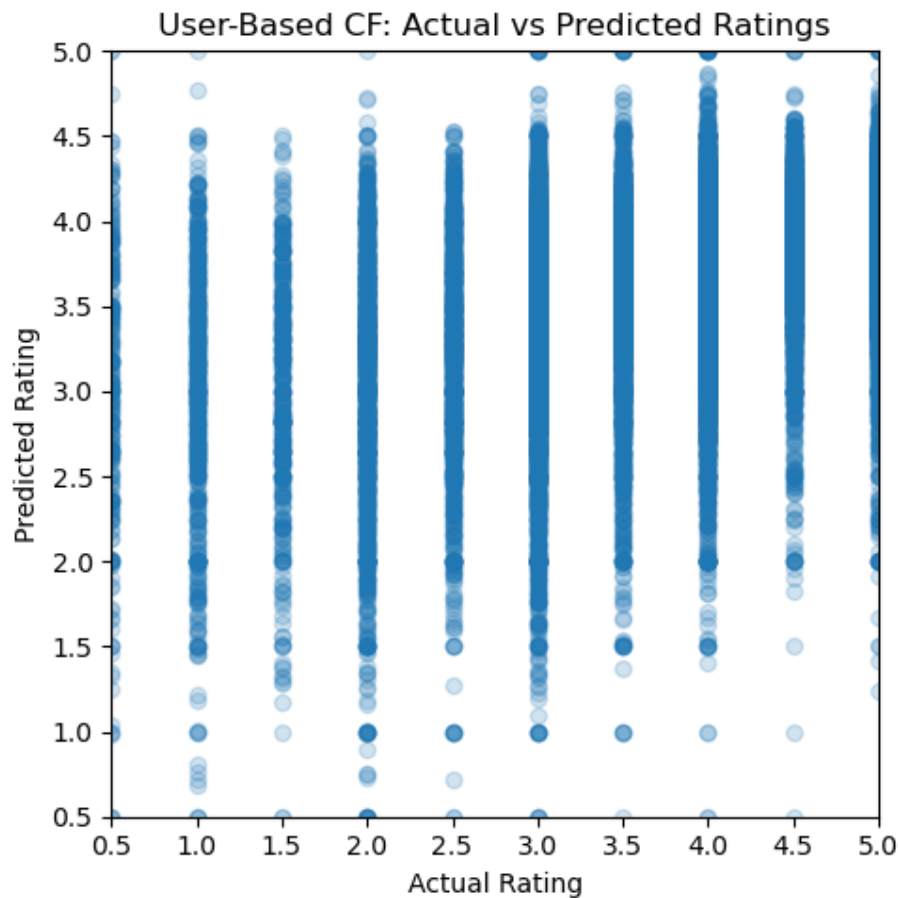


Figure 8. User-based collaborative filtering: actual vs. predicted ratings.

The user-based predictions show more spread, which matches the weaker RMSE and MAE results.

6. Results and Interpretation

The main result of the project is that item-based collaborative filtering is the strongest approach for this dataset. This was not only shown by the RMSE and MAE values, but also supported by the structure of the data. The user-item matrix is sparse, and most users have rated only a small fraction of all movies. Because of this, user-to-user similarity can be weak or noisy.

Item-based similarity was more stable because popular movies are rated by many users. This gives the algorithm more shared information to estimate movie-to-movie relationships. In this project, that stability made item-based collaborative filtering more accurate than both the baseline and the user-based model.

The result also shows why it is important to compare models instead of assuming one approach will work best. User-based collaborative filtering is intuitive because it tries to find similar people, but in this dataset it performed worse than the baseline. Item-based collaborative filtering was a better fit for the actual data.

7. Example Recommendation Outputs

The notebook also produced recommendation and similar-user outputs that were later used to support the web interface. The final web app uses item-based collaborative filtering as the main recommendation method because it performed best during testing.

Example Item-Based Recommendation	Genres	Predicted Rating
Babe: Pig in the City (1998)	Adventure Children Drama	5.000
Casino Royale (2006)	Action Adventure Thriller	5.000
Three Colors: Blue (Trois couleurs: Bleu) (1993)	Drama	5.000
Dark Knight Rises, The (2012)	Action Adventure Crime IMAX	5.000
Mystery Science Theater 3000: The Movie (1996)	Comedy Sci-Fi	5.000

Similar User ID	Similarity
368	0.2905
91	0.2866
64	0.2847
330	0.2798
202	0.2793

For the final web application demonstration, user 414 was used as a strong example because this user has many ratings. The application displayed the user rating history, recommended new movies not previously rated by the user, and listed users with similar rating behavior.

8. Web-Based System Implementation

The final project required a web-based interface. To make the project more complete, the application was implemented as a full-stack system rather than only a notebook. The stack used React + TypeScript for the frontend, Flask for the backend API, MySQL for data storage, and Docker for containerization.

Component	Technology	Purpose
Frontend	React + TypeScript + Vite	User interface for login, dashboard, recommendations, and similar users.
Backend	Flask	REST API that serves model outputs and connects the frontend to the recommender logic.
Database	MySQL	Stores cleaned ratings, movies, genre features, and merged display tables.
Containerization	Docker Compose	Runs MySQL, backend, and frontend together in a reproducible setup.

The web app satisfies all required web interface features:

Requirement	Implementation
Allows a user to login and logout	The login page accepts a MovieLens userId and the dashboard includes a logout button.
Displays items the user has rated	The dashboard displays a table of highly rated movies for the logged-in user.
Provides recommendations to items the user has not rated	The recommendation cards show movies filtered to exclude all movies already rated by the user.
Provides recommendation of similar users	The dashboard lists users with similar rating behavior and their similarity scores.

The backend API included endpoints for health checking, login/logout, sample users, rated movies, recommendations, similar users, and model summary. The recommender was trained once and then used to serve recommendation requests through the API.

9. Web Application Screenshots

The screenshots below show the final web application interface. These screenshots should be used in the final submission to demonstrate that the project includes a working web-based system.

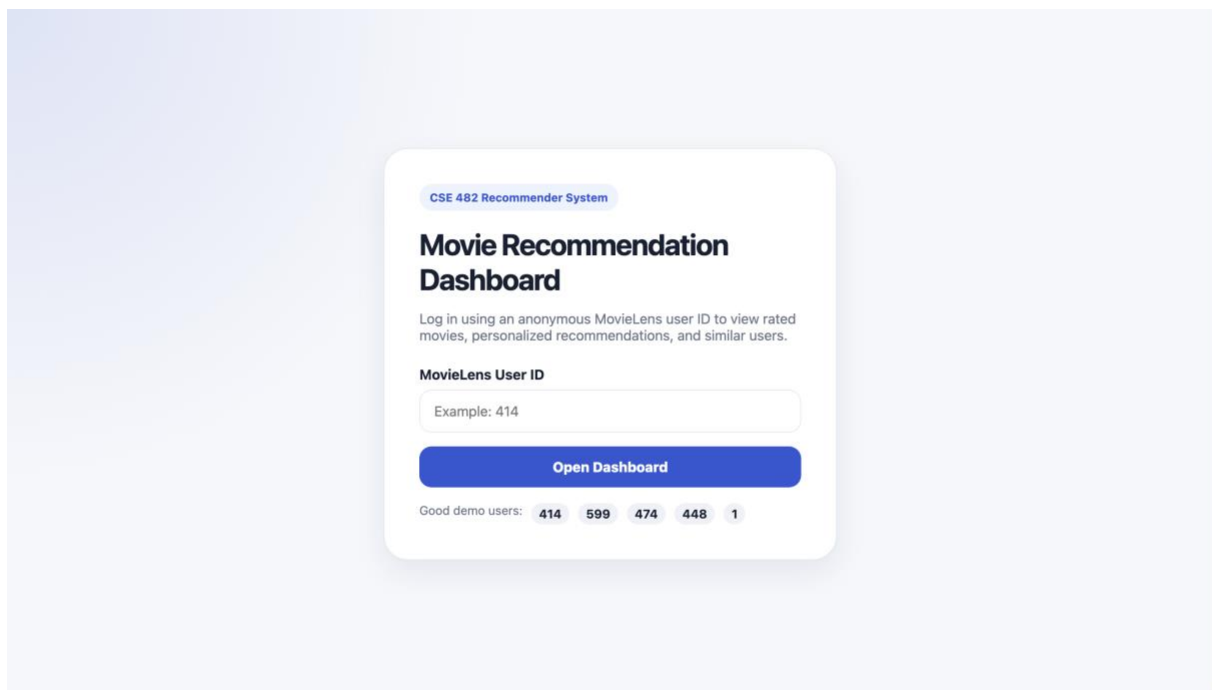


Figure 9. Login page for the MovieLens recommender dashboard.

The login page allows a user to enter a MovieLens userId. Since the MovieLens users are anonymous, the login is based on valid user IDs from the dataset.

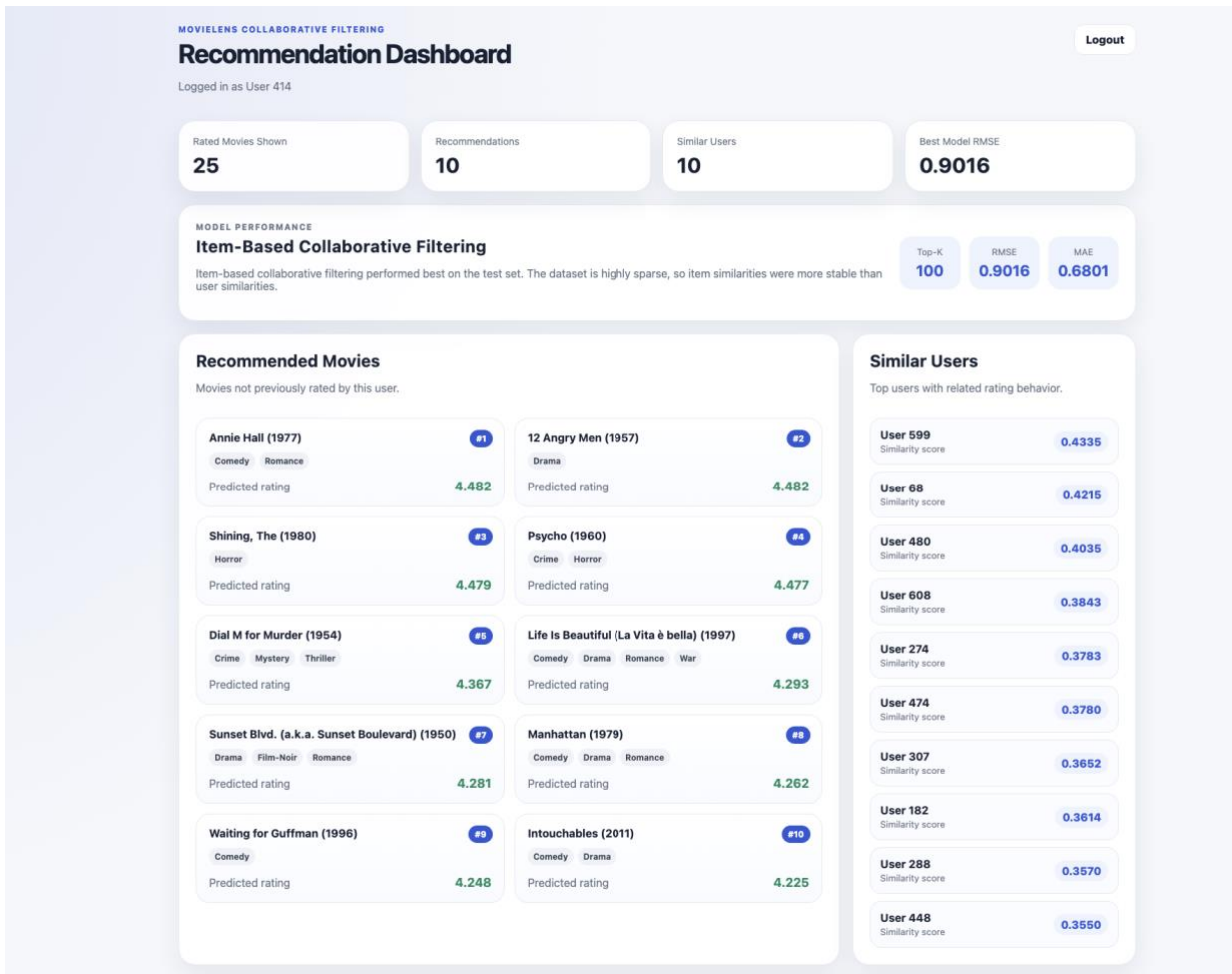


Figure 10. Dashboard showing model summary, recommendations, and similar users.

The dashboard displays model performance, the number of recommendations, the number of similar users, and the item-based recommendation results for the selected user.

Highly Rated Movies		
Movies this user rated highly in the MovieLens dataset.		
MOVIE	GENRES	RATING
Great Escape, The (1963)	Action Adventure Drama War	5
Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1964)	Comedy War	5
Princess and the Warrior, The (Krieger und die Kaiserin, Der) (2000)	Drama Romance	5
It's a Wonderful Life (1946)	Children Drama Fantasy Romance	5
2001: A Space Odyssey (1968)	Adventure Drama Sci-Fi	5
Citizen Kane (1941)	Drama Mystery	5
Roman Holiday (1953)	Comedy Drama Romance	5
Peter Pan (1953)	Animation Children Fantasy Musical	5
Casablanca (1942)	Drama Romance	5
Some Like It Hot (1959)	Comedy Crime	5
Rear Window (1954)	Mystery Thriller	5
Vertigo (1958)	Drama Mystery Romance Thriller	5
Singin' in the Rain (1952)	Comedy Musical Romance	5
Moulin Rouge (2001)	Drama Musical Romance	5
Bound (1996)	Crime Drama Romance Thriller	5
Godfather, The (1972)	Crime Drama	5
Memento (2000)	Mystery Thriller	5
Outsiders, The (1983)	Drama	5
Lost Boys, The (1987)	Comedy Horror Thriller	5
Trainspotting (1996)	Comedy Crime Drama	5
Hope and Glory (1987)	Drama	5
Garden State (2004)	Comedy Drama Romance	5
Saving Private Ryan (1998)	Action Drama War	5
Dirty Rotten Scoundrels (1988)	Comedy	5
Monty Python's Life of Brian (1979)	Comedy	5

Figure 11. Rated movies table for the selected user.

The rated movies table confirms that the application can display items already rated by the logged-in user. Recommendations are separately filtered so that already-rated movies are not shown as new recommendations.

10. Further Discussion and Lessons Learned

The most important lesson from the project is that the quality and structure of the data strongly affects recommender performance. The user-based method seems reasonable at first because it tries to find similar people, but the sparse rating matrix made those similarities less reliable. The item-based model worked better because movie-to-movie similarities were more stable.

A second lesson is that model accuracy is only one part of a complete system. A notebook can show RMSE and MAE, but a real application also needs a database, API endpoints, frontend state management, filtering logic, and a usable interface. Connecting these pieces took additional work beyond the model itself.

A third lesson is that small implementation details matter. One example was making sure the recommendations exclude every movie the user already rated, not just movies in the training set. Without that fix, the app could recommend movies the user had already seen, which would violate the project requirement. Fixing that made the system logically correct.

Training efficiency was affected by sparsity, the number of movies, and the value of k . Increasing k from 50 to 100 improved the results, but increasing k also increases computation because more neighbors are considered. For this dataset, $k = 100$ was a good balance between performance and complexity.

11. Future Work

Several improvements could be made if this project were extended:

- Use a hybrid recommender that combines collaborative filtering with genre information.
- Test matrix factorization methods such as SVD to better handle sparse data.
- Add more movie metadata such as actors, directors, release year, and tags.
- Add UI filters so users can filter recommendations by genre or minimum predicted rating.
- Store precomputed recommendations in the database to reduce runtime computation.
- Deploy the frontend to Google Cloud and the backend/API to a cloud service such as Google Cloud Run.
- Add pagination and search to the rated movies table for users with many ratings.

12. Final Conclusion

This project successfully developed a complete movie recommendation system using the MovieLens latest-small dataset. The project included data collection, preprocessing, MySQL storage, exploratory analysis, collaborative filtering model comparison, interpretation of results, and a working web application.

The best performing model was item-based collaborative filtering, which achieved $RMSE = 0.9016$ and $MAE = 0.6801$ with $k = 100$. This model performed better than both the baseline and the user-based collaborative filtering model. The main reason was the high sparsity of the dataset, which made user similarity less reliable than item similarity.

The final application met the project requirements by allowing login/logout, displaying rated movies, providing recommendations for unrated movies, and showing similar users. Overall, the project combined data analysis and software engineering into one complete recommender system.

Reference

Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4), 19:1-19:19. <https://doi.org/10.1145/2827872>